TIP102 | Intermediate Technical Interview Prep

Intermediate Technical Interview Prep Fall 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: **134071**

Need help? Post on our **class slack channel** or email us at **support@codepath.org**

Getting Started

Learning with AI ✨

▶ Course Info

Course Progress

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

Week 11

Week 12

Week 7: Session 1

Binary Search | Recursion II

In this session, students will dive deep into the concept of recursion, a fundamental programming technique used to solve problems by breaking them down into simpler, self-similar subproblems. The session will cover how to write recursive functions - specifically how to identify the base case and the importance of recursive calls, equipping students with the skills needed to tackle recursive programming questions.

> You can find session recordings and more on the resources tab. Session slide decks are available on the overview tab.

🎢 Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.

👷 Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
    - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like VSCode, PyCharm, Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

▶ **Note on Expectations**

🔎 Problem Solving Approach

We will approach problems using the six steps in the UMPIRE approach.

UMPIRE: Understand, Match, Plan, Implement, Review, Evaluate.

We'll apply these six steps to the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem
- **Match** identifies common approaches you've seen/used before
- **Plan** a solution step-by-step, and
- **Implement** the solution
- **Review** your solution
- **Evaluate** your solution's time and space complexity and think critically about the advantages and disadvantages of your chosen approach.

Problem Set Version 1

▼ Problem 1: Counting Iron Man's Suits

Tony Stark, aka Iron Man, has designed many different suits over the years. Given a list of strings `suits` where each string is a suit in Stark's collection, count the total number of suits in the list.

1. Implement the solution *iteratively* without the use of the `len()` function.
2. Implement the solution *recursively*.
3. Discuss: what are the similarities between the two solutions? What are the differences?

```
def count_suits_iterative(suits):
    pass

def count_suits_recursive(suits):
    pass
```

Example Usage:

```
print(count_suits_iterative(["Mark I", "Mark II", "Mark III"]))
print(count_suits_recursive(["Mark I", "Mark II", "Mark III"]))
```

Example Output:

```
3
3
```

▶ 💡 **Hint: Recursion**

Close Section

▼ Problem 2: Collecting Infinity Stones

Thanos is collecting Infinity Stones. Given an array of integers `stones` representing the power of each stone, return the total power using a recursive approach.

Evaluate the time complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time complexity.

```
def sum_stones(stones):
    pass
```

Example Usage:

```
print(sum_stones([5, 10, 15, 20, 25, 30]))
print(sum_stones([12, 8, 22, 16, 10]))
```

Example Output:

```
105
68
```

Close Section

▼ Problem 3: Counting Iron Man's Unique Suits

Some of Iron Man's suits are duplicates. Given a list of strings `suits` where each string is a suit in Stark's collection, count the total number of *distinct* suits in the list.

1. Implement the solution *iteratively*.
2. Implement the solution *recursively*.
3. Discuss: what are the similarities between the two solutions? What are the differences?

4. Evaluate the time complexity of each solution. Are they the same? Define your variables and provide a rationale for why you believe your solution has the stated time complexity.

```
def count_suits_iterative(suits):
    pass

def count_suits_recursive(suits):
    pass
```

Example Usage:

```
print(count_suits_iterative(["Mark I", "Mark I", "Mark III"]))
print(count_suits_recursive(["Mark I", "Mark I", "Mark III"]))
```

Example Output:

```
2
2
```

▶ 💡 **Hint: Multiple Recursive Cases**

Close Section

▼ Problem 4: Calculating Groot's Growth

Groot grows according to a pattern similar to the Fibonacci sequence. Given `n`, find the height of Groot after `n` months using a recursive method.

The Fibonacci numbers, commonly denoted `F(n)` form a sequence, called the Fibonacci sequence, such that each number is the sum of the two preceding ones, starting from `0` and `1`. That is,

```
F(0) = 0, F(1) = 1
F(n) = F(n - 1) + F(n - 2), for n > 1.
```

Evaluate the time complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time complexity.

```
def fibonacci_growth(n):
    pass
```

Example Usage:

```
print(fibonacci_growth(5))
print(fibonacci_growth(8))
```

Example Output:

```
5
21
```

Close Section

▼ Problem 5: Calculating the Power of the Fantastic Four

The superhero team, The Fantastic Four, are training to increase their power levels. Their power level is represented as a power of 4. Write a recursive function that calculates the result of 4 raised to the nth power to determine their training level.

Evaluate the time complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time complexity.

```
def power_of_four(n):
    pass
```

Example Usage:

```
print(power_of_four(2))
print(power_of_four(-2))
```

Example Output:

```
16
Example 1 Explanation: 4 to the 2nd power (4 * 4) is 16.
.0625
Example 2 Explanation: 4 to the power of -2 is 1/(4 * 4), which is 0.0625.
```

Close Section

▼ Problem 6: Strongest Avenger

The Avengers need to determine who is the strongest. Given a list of their strengths, find the maximum strength using a recursive approach without using the `max()` function.

Evaluate the time complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time complexity.

```
def strongest_avenger(strengths):
    pass
```

Example Usage:

```
print(strongest_avenger([88, 92, 95, 99, 97, 100, 94]))
print(strongest_avenger([50, 75, 85, 60, 90]))
```

Example Output:

```
100
Example 1 Explanation: The maximum strength among the Avengers is 100.

90
Example 2 Explanation: The maximum strength among the Avengers is 90.
```

Close Section

▼ Problem 7: Counting Vibranium Deposits

In Wakanda, vibranium is the most precious resource, and it is found in several deposits. Each deposit is represented by a character in a string (e.g., `"V"` for vibranium, `"G"` for gold, etc.)

Given a string `resources`, write a recursive function `count_deposits()` that returns the total number of distinct *vibranium* deposits in `resources`.

Evaluate the time complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time complexity.

```
def count_deposits(resources):
    pass
```

Example Usage:

```
print(count_deposits("VVVVV"))
print(count_deposits("VXVYGA"))
```

Example Output:

```
5
2
Example 2 Explanation: There are two characters "V" in the string "VXVYGA",
therefore there are two vibranium deposits in the string.
```

Close Section

▼ Problem 8: Merging Missions

The Avengers are planning multiple missions, and each mission has a priority level represented as a node in a linked list. You are given the heads of two sorted linked lists, `mission1` and `mission2`, where each node represents a mission with its priority level.

Implement a recursive function `merge_missions()` which merges these two mission lists into one sorted list, ensuring that the combined list maintains the correct order of priorities. The merged list should be made by splicing together the nodes from the first two lists.

Return the head of the merged mission linked list.

```
class Node:
  def __init__(self, value, next=None):
      self.value = value
      self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def merge_missions(mission1, mission2):
    pass
```

Example Usage:

```
mission1 = Node(1, Node(2, Node(4)))
mission2 = Node(1, Node(3, Node(4)))

print_linked_list(merge_missions(mission1, mission2))
```

Example Output:

```
1 -> 1 -> 2 -> 3 -> 4 -> 4
```

Close Section

▼ Problem 9: Merging Missions II

Below is an iterative solution to the `merge_missions()` function from the previous problem. Compare your recursive solution to the iterative solution below.

Discuss with your podmates. Which solution do you prefer?

```
class Node:
    def __init__(self, value=0, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def merge_missions_iterative(mission1, mission2):
    temp = Node()  # Temporary node to simplify the merging process
    tail = temp
```

```
    while mission1 and mission2:
        if mission1.value < mission2.value:
            tail.next = mission1
            mission1 = mission1.next
        else:
            tail.next = mission2
            mission2 = mission2.next
        tail = tail.next

    # Attach the remaining nodes, if any
    if mission1:
        tail.next = mission1
    elif mission2:
        tail.next = mission2

    return temp.next  # Return the head of the merged linked list
```

Close Section

Problem Set Version 2

- ▶ Problem 1: Calculating Village Size
- ▶ Problem 2: Counting the Castle Walls
- ▶ Problem 3: Reversing a Scroll
- ▶ Problem 4: Palindromic Name
- ▶ Problem 5: Doubling the Power of a Spell
- ▶ Problem 6: Checking the Knight's Path
- ▶ Problem 7: Finding the Longest Winning Streak
- ▶ Problem 8: Weaving Spells
- ▶ Problem 9: Weaving Spells II